Comunicación entre procesos: Introducción

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Definición

- La **comunicación entre procesos** (*IPC*, del inglés Inter-Process Communication) es una función clave de los SO multiusuarios y multitarea que permite que los procesos se coordinen de manera eficiente y controlada.

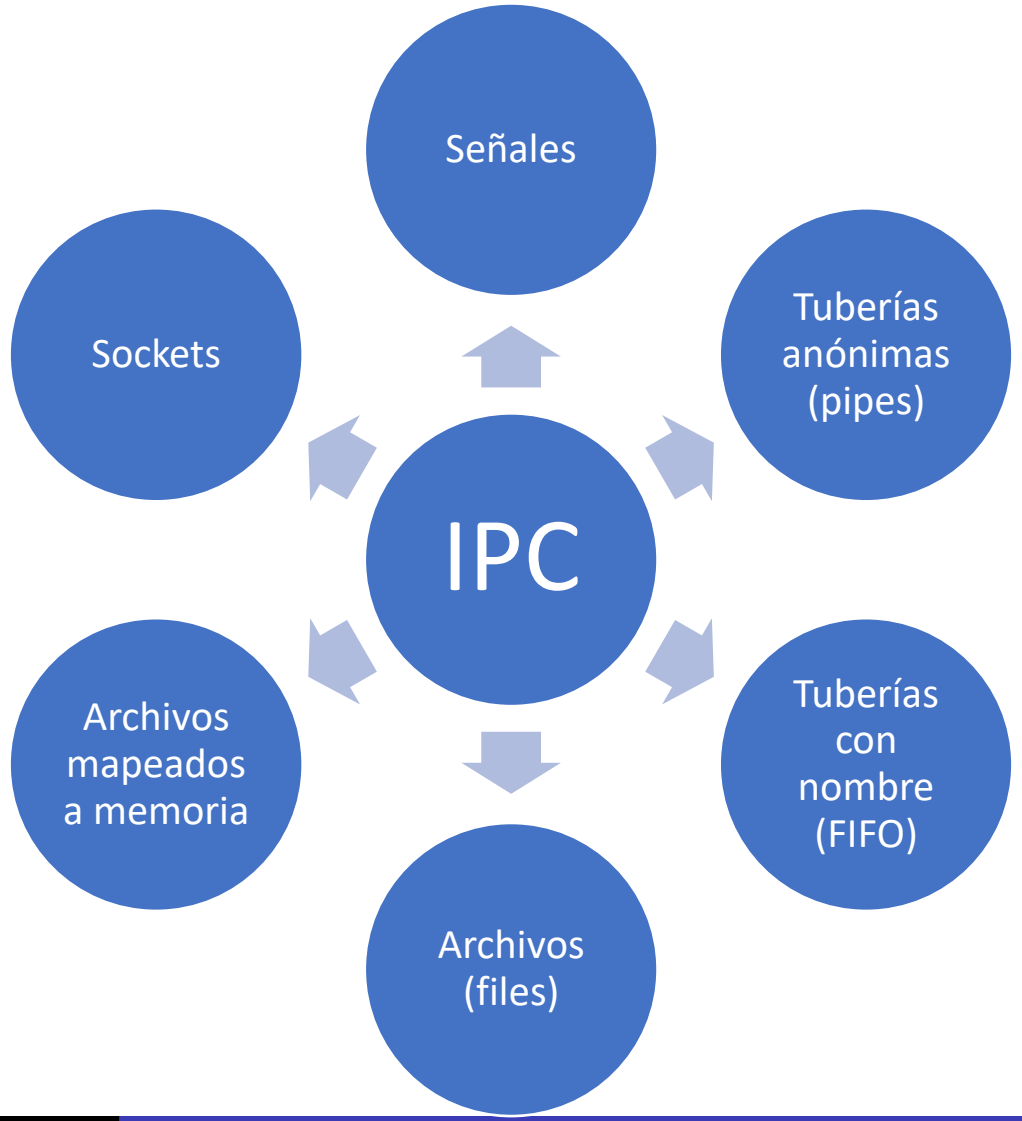
¿Por qué es necesaria?

- Porque permite que los procesos en un SO interactúen y compartan información entre sí.
- Por ejemplo, un proceso puede necesitar información de otro proceso para completar una tarea o puede necesitar enviar información a otro proceso para que este pueda continuar con su ejecución.
- ¿Qué nos permite el ICP?:
  - Aceleración computacional
  - Modularidad
  - Intercambio de información y datos
  - Separación de privilegios
  - Los procesos pueden comunicarse entre sí y sincronizar su acción.

Ejemplos

Ventaja	Ejemplo
Coordinación y colaboración efectiva entre procesos	Un proceso de edición de video puede enviar información a un proceso de renderizado para que este pueda generar una vista previa en tiempo real.
Compartir información de manera eficiente y controlada	Un proceso de base de datos puede enviar información a un proceso de visualización para que este pueda mostrar los datos en una tabla o gráfico.
Permitir que el SO funcione de manera eficiente	Un proceso de gestión de memoria puede enviar información a un proceso de ejecución para que este pueda liberar memoria cuando sea necesario.

Tipos de comunicación entre procesos

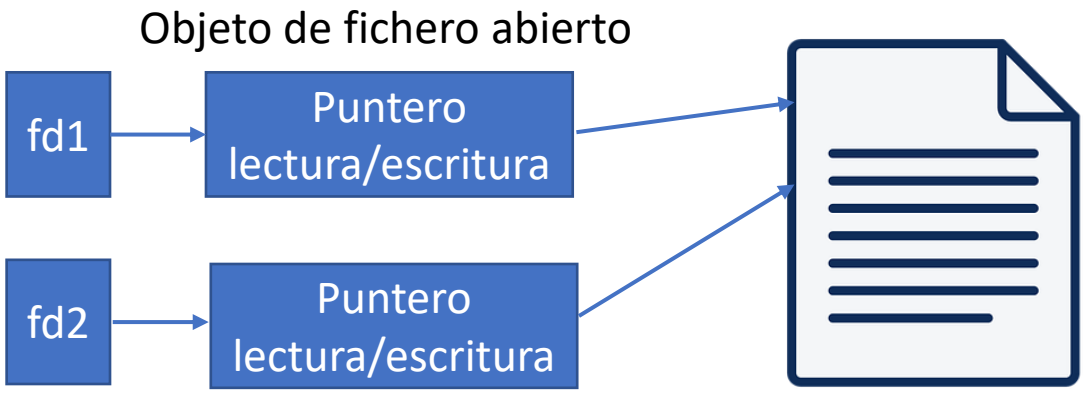


Ficheros

- Para POSIX, un **fichero** es una secuencia de bytes organizados en un dispositivo de almacenamiento (red, disco, flash USB, tarjeta SD, ….)
- Los ficheros se tratan de manera uniforme, independientemente del tipo de dispositivo en el que estén almacenados.
- En UNIX, los ficheros son accedidos secuencialmente por defecto. Cuando un usuario abre el fichero, el núcleo inicializa el puntero de lectura/escritura a cero. Cada vez que el proceso lee o escribe datos, el núcleo avanza el puntero en la cantidad de bytes transferidos.
- Todos los ficheros abiertos por un proceso tienen asociado un **descriptor de fichero fd** que asigna el núcleo del SO.

Descriptor de ficheros

- fd representa un número entero (no negativo) que identifica de manera única el fichero que se está manipulando
- En el contexto de SO, el *fd* un **índice de un vector** que contiene la información de los ficheros abiertos.
  - 0 -> Fichero estándar de entrada (stdin) – por lo general el teclado
  - 1 -> Fichero estándar de salida (stout) – por lo general la pantalla
  - 2 -> Fichero estándar de salida de mensajes de error (stderr) – pantalla



Conceptos introductorios

- El SO nos ofrece llamadas al sistema (que devuelven un *fd*) para realizar comunicaciones entre procesos.
  - **Abrir una comunicación**
    - `open()` para abrir un fichero de disco
    - `pipe()` para crear una tubería (anónima)
    - `mkfifo()` para crear una tubería (con nombre)
    - `socket()` para crear una conexión (remota) via red
    - …
  - **Enviar y recibir información** (tienen *fd* como primer argumento)
    - `write()`
    - `read()`
  - **Cerrar una comunicación** (único argumento: *fd*)
    - `close()`

Gracias

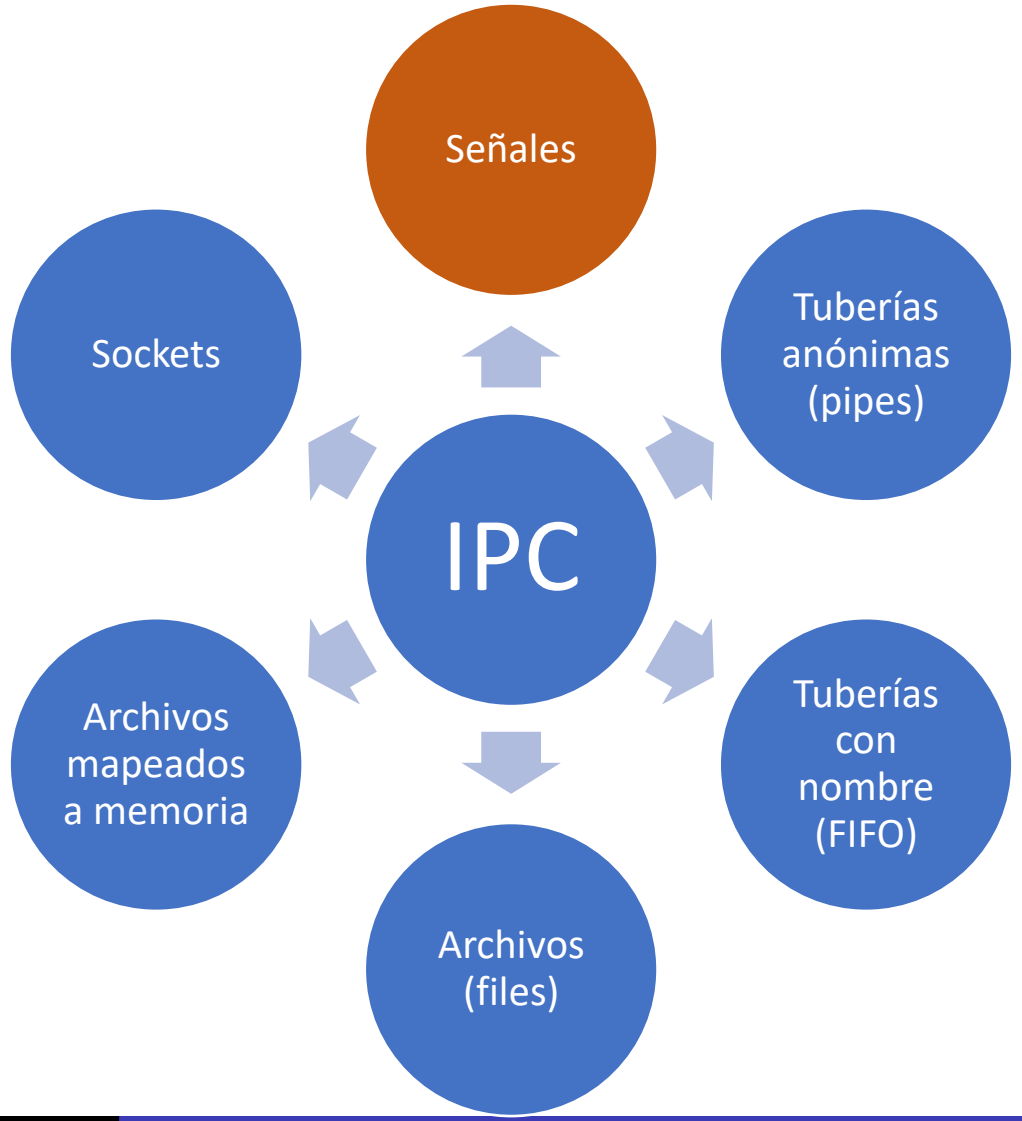
Comunicación entre procesos: Señales

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Tipos de comunicación entre procesos



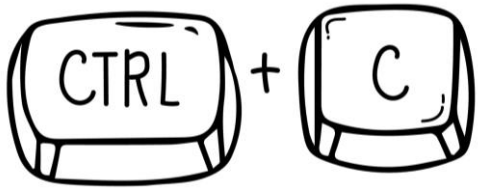
¿Qué son las señales?

- Mecanismo de **comunicación asíncrona** entre procesos (necesita PID del proceso receptor) y/o entre el núcleo y los procesos.
- Es una notificación que indica a un proceso que ha ocurrido un evento (provocado por otro proceso o por el SO) en el sistema.
- Al enviar una señal, el núcleo interrumpe la ejecución del proceso y se llama una determinada función (previamente indicada por el proceso).
- El proceso puede:
  - Terminar su ejecución
  - Ignorar la señal
  - Capturarla y manejarla
  - Detenerse y esperar interacción del usuario

Ejemplos de uso de las señales

- Al producirse una excepción el SO envía una señal al proceso que ha provocado la excepción. El proceso puede enviar información a un fichero log del problema antes de “morir”.

- Al pulsar CTRL+C en un terminal, el SO realiza una notificación al proceso que se quiere parar. En ese momento el proceso puede guardar a disco todos los datos que tiene antes de “morir”.



- En los Sistemas de Alimentación Ininterrumpida (SAI), el sistema puede realizar una notificación asíncrona a los procesos si le queda poca batería. Así los procesos pueden guardar datos a disco antes de que se apague el sistema.

Señales (POSIX) más comunes

- **SIGKILL**: Señal para matar un proceso inmediatamente. El proceso no puede capturar o ignorar esta señal (**No se puede ignorar**).
- **SIGTERM**: Señal para terminar un proceso de manera controlada. El proceso puede capturar y manejar esta señal.
- **SIGSTOP**: Señal para detener un proceso temporalmente. El proceso no puede capturar o ignorar esta señal (**No se puede ignorar**).
- **SIGCONT**: Señal para reanudar un proceso detenido anteriormente con SIGSTOP.
- **SIGINT**: Señal para interrumpir la ejecución de un proceso desde el teclado.

https://en.wikipedia.org/wiki/Unix_signal

¿Qué acciones pueden producirse al recibir una señal?

- Cada proceso indica al SO que hacer en caso de recibir una señal determinada.
  - **Acción por defecto** (en caso de que no se indique nada). Ver https://en.wikipedia.org/wiki/Unix_signal
  - **Capturar la señal**. El proceso indica al SO que función tiene que llamar en producirse un determinado evento.
  - **Ignorar la señal**. El proceso puede indicar al SO que ignore la señal. Puede ser peligroso (para el proceso) en caso de que, por ejemplo, se produzca una división por cero y se continue la ejecución normalmente.

Código

- exemple_sigterm.c

```
int done = 0;

void finalitzar(int signo)
{
  printf("He capturat el senyal.\n");
  done = 1;
}

int main(void)
{
  int comptador = 0;

  signal(SIGTERM, finalitzar);

  while (!done)
  {
    sleep(1);
    comptador++;
    printf("He esperat %d segons\n", comptador);
  }

  printf("S'ha acabat el bucle. Finalitzo normalment\n");

  return 0;
}
```

`signal()` se utiliza en Unix para establecer el comportamiento que un proceso debe tomar en respuesta a una señal específica enviada por el SO o por otro proceso.

Al código anterior…

- Compila y ejecuta el código en un terminal UNIX
- Desde un nuevo terminal, mata el proceso con el comando `kill` (por defecto `kill` envia la señal **SIGTERM** al proceso indicado)
  - `kill <pid>`
  - PID
    - `ps aux | grep exemple`
    - `top -p $(pgrep exemple)`

- Ahora os propongo realizar los cambios en el código.
  - Al presionar "CTRL+C" en un terminal, el SO envía la señal **SIGINT**. Modifica el código anterior para que se llame a la función `finalizar` al presiona éstas teclas.
  - ¿Qué cambios son necesarios para capturar pero ignorar la señal SIGTERM/SIGINT?. Pista: revisar la condición para salir del bucle while.

Señales disponibles (SIGKILL)

- Podemos enviar cualquier señal con el comando kill
  - `kill -l`

```
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

- La señal **SIGKILL** es similar a SIGTERM pero con una diferencia: el SIGKILL no se puede capturar como lo hemos hecho con SIGTERM. Es un método a prueba de fuego para matar un proceso en caso de que no nos haga caso.
  - `kill -SIGKILL <pid>`

Señales disponibles (SIGSTOP, SIGCONT)

- **SIGSTOP**: Señal para detener un proceso temporalmente. El proceso no puede capturar o ignorar esta señal (No se puede ignorar).
  - `kill -SIGSTOP <pid>`

- **SIGCONT**: Señal para reanudar un proceso detenido anteriormente con SIGSTOP.
  - `kill -SIGCONT <pid>`

- Ejercicio:
  - Abrir un navegador (firefox) y buscar su PID
  - `kill -SIGSTOP <pid>`
  - Probar a realizar alguna acción en el navegador
  - `kill -SIGCONT <pid>`

Señales disponibles (SIGUSR1, SIGUSR2)

- **SIGUSR1, SIGUSR2**: Señales definidas por el usuario para ser usada en programas de aplicación.
- Ver sigusr.c

```
void sigusr(int signo)
{
  if (signo == SIGUSR1)
    printf("He rebut SIGURS1.\n");
  if (signo == SIGUSR2)
    printf("He rebut SIGURS2.\n");
}

int main(void)
{
  signal(SIGUSR1, sigusr);
  signal(SIGUSR2, sigusr);

  while (1)
  {
    printf("Espero senyal!\n");
    pause();
  }

  return 0;
}
```

`pause()` bloquea el proceso hasta que recibe una señal.

Al código anterior

- Ejecutar sigusr.c (el proceso se queda esperando una señal)
- Ejecutar enviar_sigusr.c (la función `kill` envia una señal)

Proceso 1 (sigusr)

```
Espero senyal!
He rebut SIGURS1.
Espero senyal!
He rebut SIGURS2.
Espero senyal!
He rebut SIGURS1.
Espero senyal!
He rebut SIGURS2.
Espero senyal!
He rebut SIGURS1.
Espero senyal!
He rebut SIGURS2.
Espero senyal!
He rebut SIGURS1.
Espero senyal!
He rebut SIGURS2.
Espero senyal!
Terminated
```

Proceso 2 (enviar_sigusr)

```
Enviare senyals al proces amb pid 10534
Envio el SIGUSR1
Envio el SIGUSR2
Envio el SIGUSR1
Envio el SIGUSR2
Envio el SIGUSR1
Envio el SIGUSR2
Envio el SIGUSR1
Envio el SIGUSR2
Envio el SIGTERM
```

Gracias

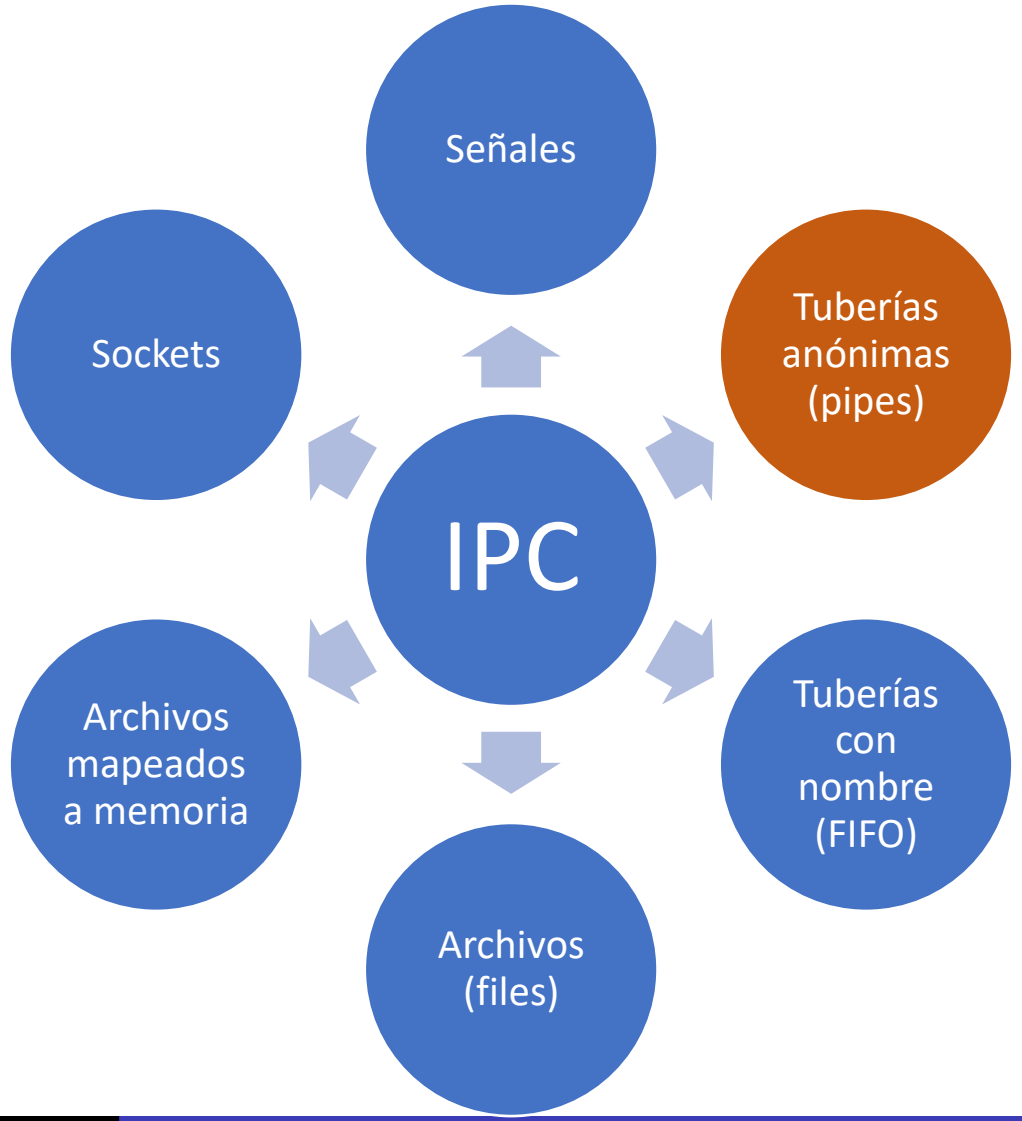
Comunicación entre procesos: Tuberías

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

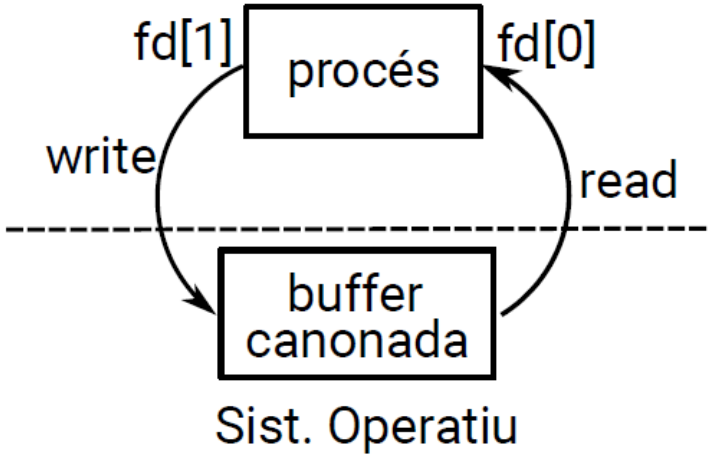
Tipos de comunicación entre procesos



Tuberias (anónimas)

- La **tubería** (pipe) es un mecanismo de comunicación entre procesos que permite la transferencia de datos entre un proceso emisor y un receptor.
- ¿Cómo lo implementamos? Llamada al sistema `pipe()` que crea:
  - Un buffer al SO que gestiona la comunicación
  - Dos descriptores de ficheros:
    - Uno de escritura
    - Uno de lectura

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
  int fd[2];

  if (pipe(fd) == -1) {
    fprintf(stderr, "pipe not created");
    exit(1);
  }

  // fd[1] per escriptura
  // fd[0] per lectura

  return 0;
}
```



Tuberias (anónimas)

- Se puede enviar cualquier tipo de datos por una tubería. Ver pipe.c

```c
int main(void)
{
  int fd[2];
  char buf[30];

  if (pipe(fd) == -1) {
    fprintf(stderr, "pipe not created");
    exit(1);
  }

  printf("writing to file descriptor #%d\n", fd[1]);
  write(fd[1], "test", 4);
  printf("reading from file descriptor #%d\n", fd[0]);
  read(fd[0], buf, 4);
  buf[4] = 0;
  printf("read \"%s\"\n", buf);

  return 0;
}
```

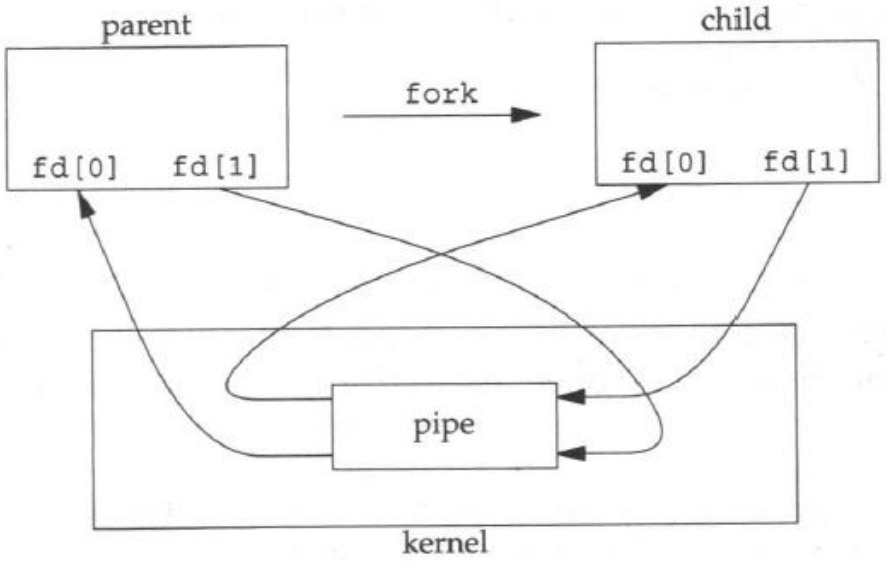
En este ejemplo, antes de imprimir la cadena recibida, hay que poner un byte 0 al final de la cadena (5º byte) ya que el `printf` imprime hasta que encuentra dicho byte 0.

Tuberias (anónimas)

- Las tuberías es canal de comunicación unidireccional entre dos procesos, siempre y cuando estos procesos tengan **relación padre-hijo**.

- P.ej., si el padre escribe a fd[1], el hijo lo podrá leer a través de f[0].

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
  int fd[2];

  pipe(fd);

  if (fork() == 0) { // child
    ...
  } else { // parent
    ...
  }

  return 0;
}
```



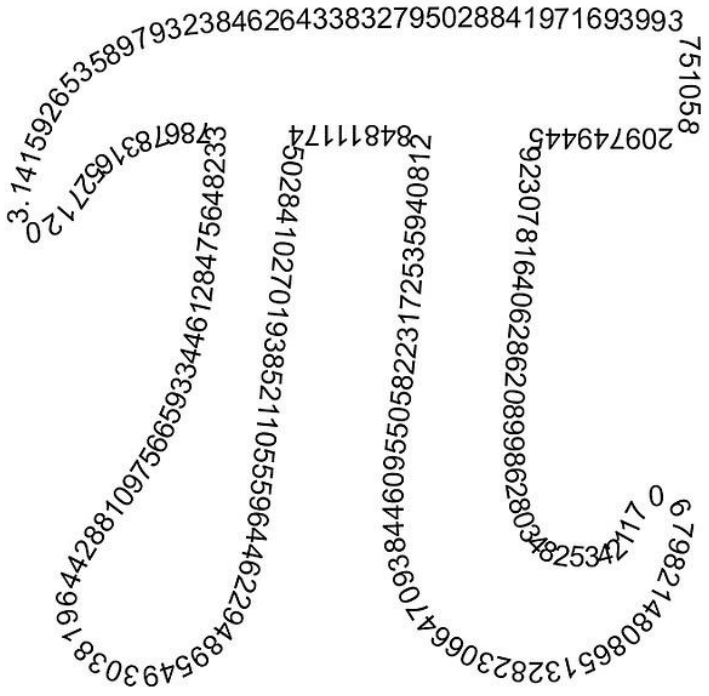
Ejemplo de tubería: búfer escritura/lectura

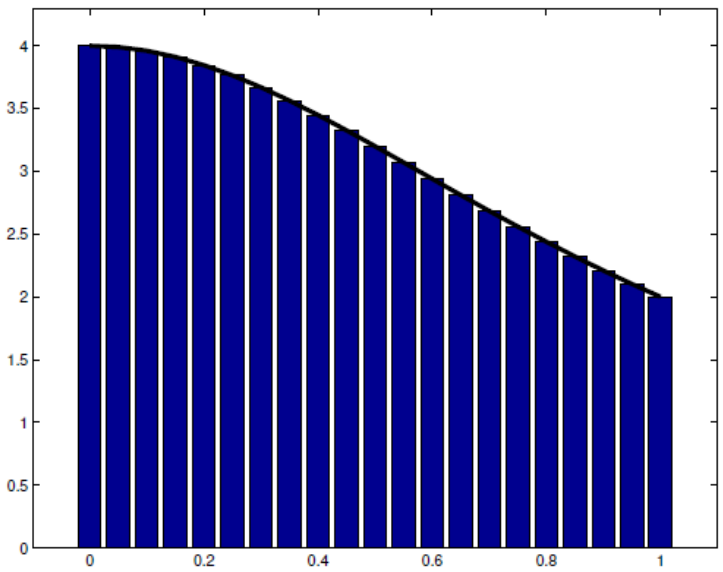
- Ver código fork-pipe.c (Por simplicidad, no se incluye instrucciones para cerrar las conexiones de las tuberías que no se utilizan)

```c
int main(void)
{
  int fd[2];
  char buf[30];

  pipe(fd);

  if (fork() == 0) { // child
    printf("child writing to file descriptor #%d\n", fd[1]);
    write(fd[1], "test", 4);
    exit(0);
  } else { // parent
    printf("parent reading from file descriptor #%d\n", fd[0]);
    read(fd[0], buf, 4);
    buf[4] = 0;
    printf("parent read \"%s\"\n", buf);
    wait(NULL);
  }

  return 0;
}
```

Ejemplo de tubería: Cálculo del valor de π

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$



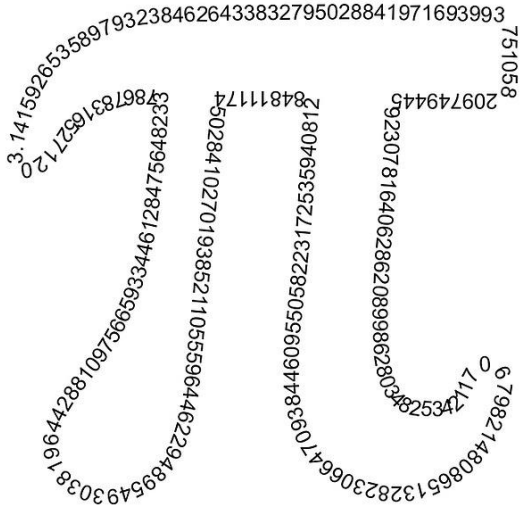


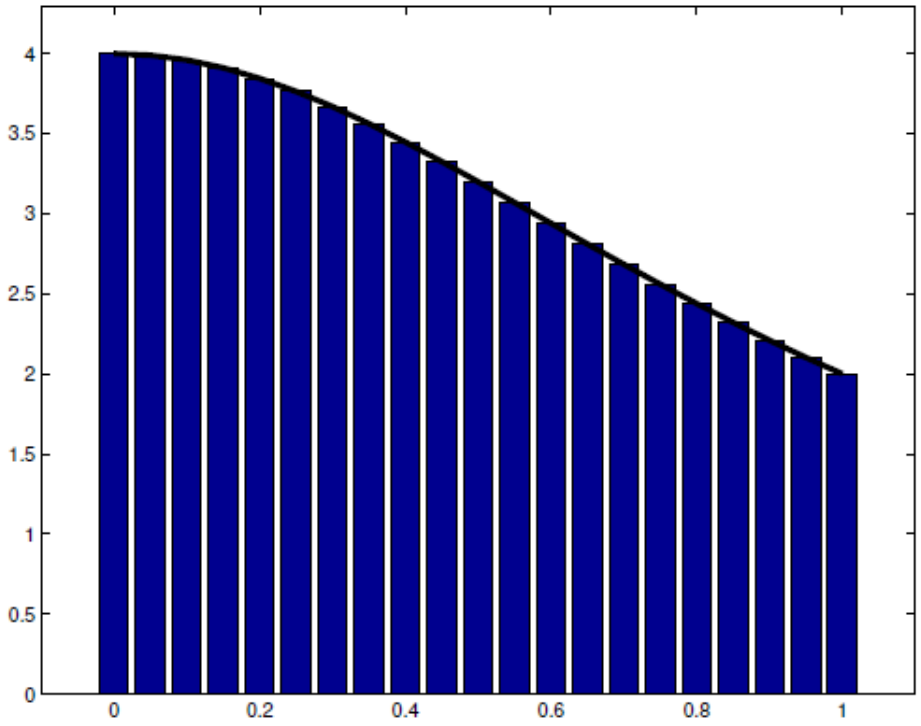

```
#define NUM_RECTS 1E09
int main()
{
  int i;
  double mid, height, width, sum = 0.0;
  double area;

  width = 1.0 / (double) NUM_RECTS;
  for(i = 0; i < NUM_RECTS; i++) {
    mid = (i + 0.5) * width;
    height = 4.0 / (1.0 + mid * mid);
    sum += height;
  }
  area  = width * sum;
  printf("pi = %e\n", area);

  return 0;
}
```

- El cálculo del área de cada rectángulo es independiente del resto de rectángulos.

Ejemplo de tubería: Cálculo del valor de π

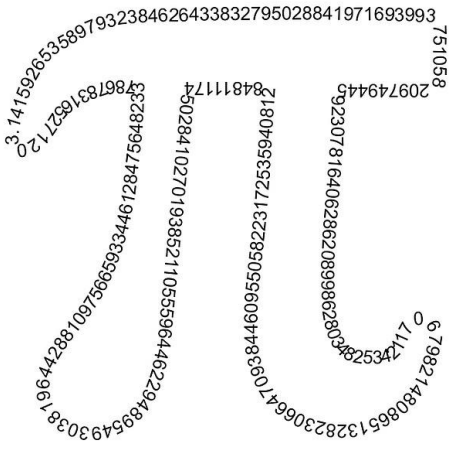
- Implementar el código (calcul_pi_1proces.c) utilizando un solo proceso
  - `time calcul_pi_1proces`
- Implementar el código utilizando dos procesos y una tubería para comunicar los resultados parciales (calcul_pi_2processos.c)
  - `time calcul_pi_2processos`





Ejemplo de tubería: Cálculo del valor de π

- El proceso padre realiza el cálculo de los rectángulos impares.
- El proceso hijo realiza el cálculo de los rectángulos pares.
- El proceso hijo comunica al padre su resultado.
- Si se dispone de 2 (o más) procesadores, el tiempo de ejecución del código con dos procesos es menor que para uno. El SO ejecuta cada proceso en CPUs diferentes.
- Este ejemplo es algo “forzado” ya que seria más práctico utilizar hilos como veremos en Sistemas Operativos 2.



Gracias

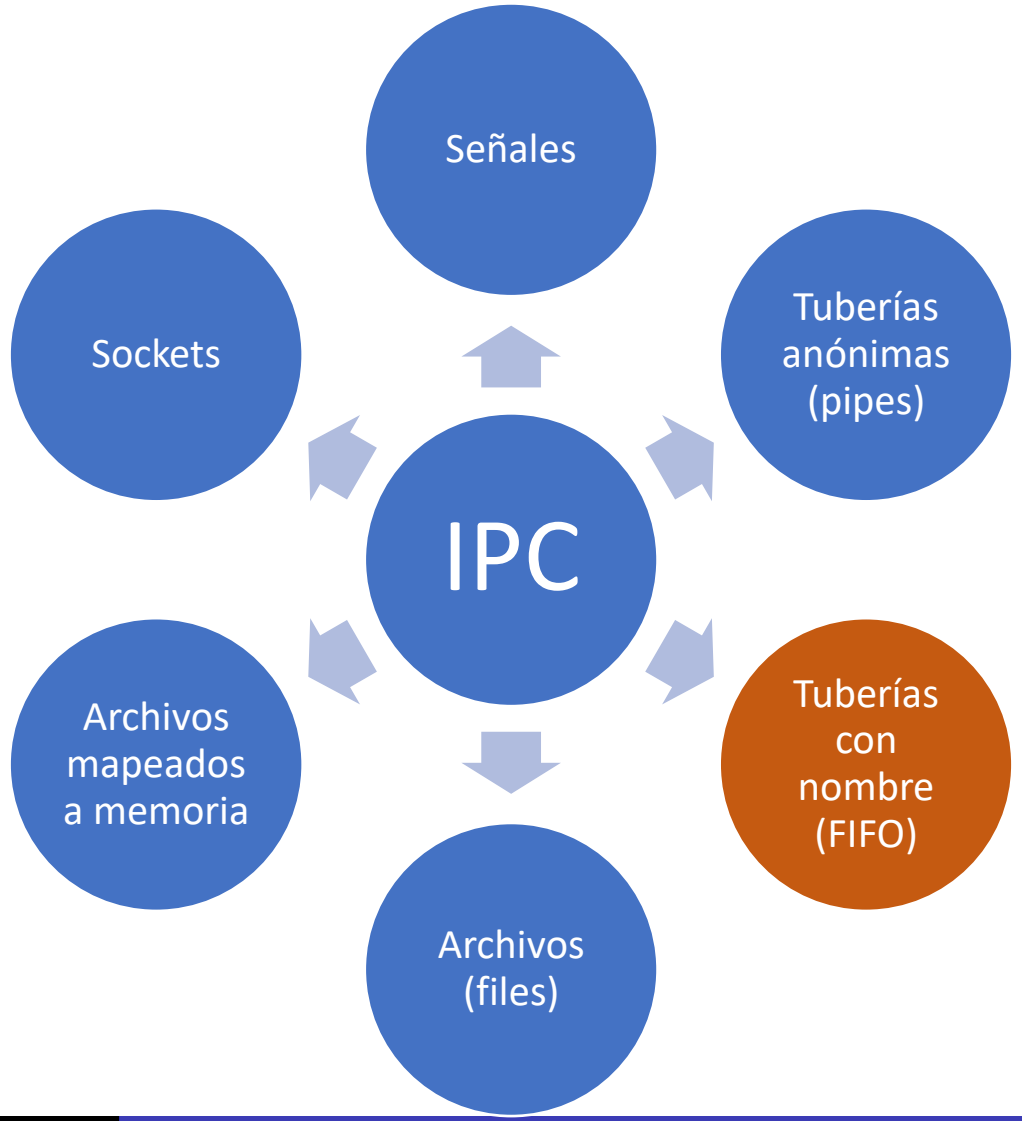
Comunicación entre procesos: Tuberías con nombre

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Tipos de comunicación entre procesos



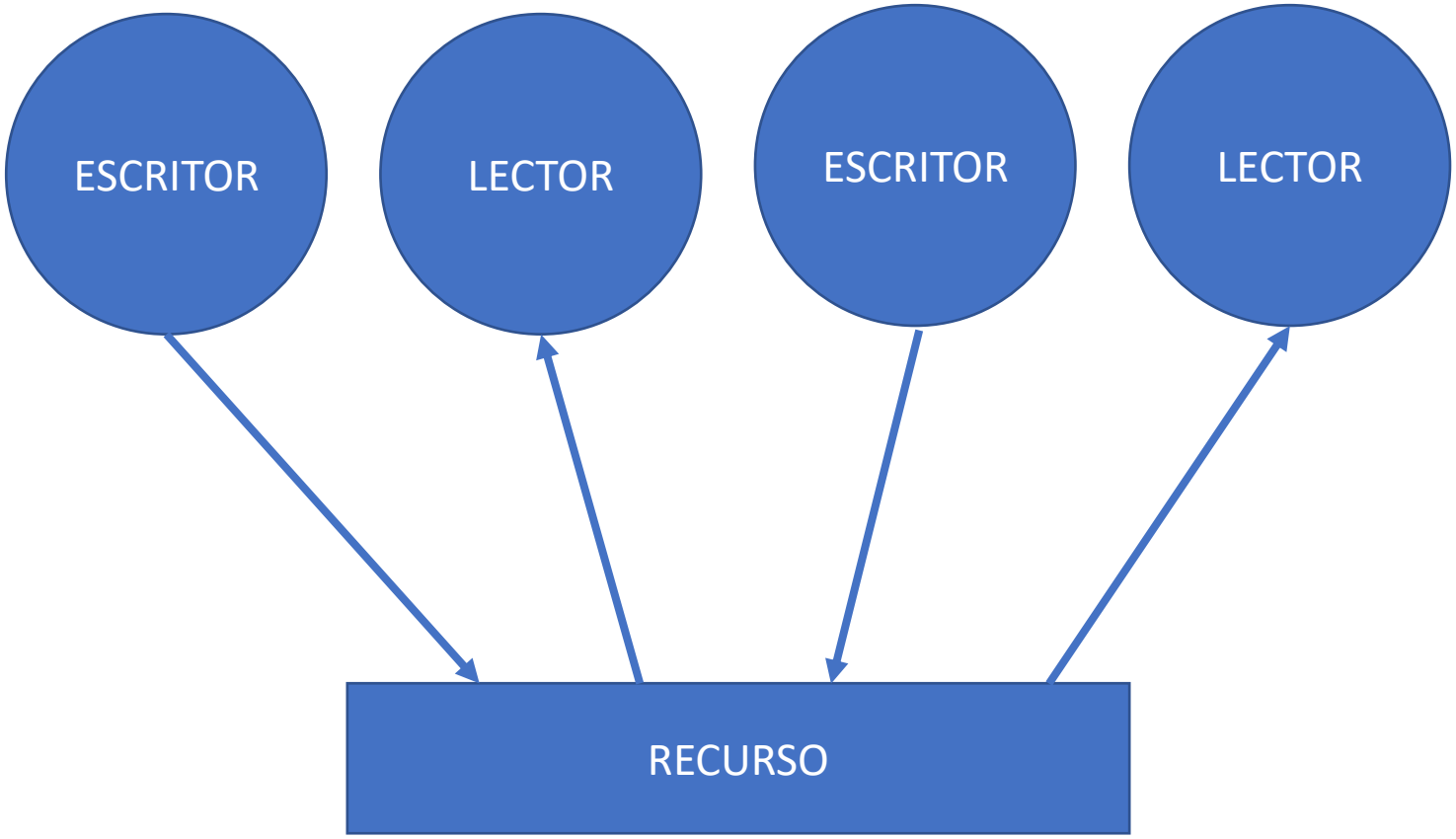
Definición

- Las **tuberías anónimas** tienen una gran desventaja
  - Solo permiten la comunicación entre procesos con relación padre-hijo
- Las **tuberías con nombre** (también llamadas tuberías FIFO; First-In-First-Out) combinan características de fichero de disco y tubería.
  - Se crean con anticipación y tienen un nombre asociado en el sistema de archivos.
  - Los procesos pueden abrir la tubería con nombre como si fuera un archivo normal, y pueden enviar o recibir datos a través de ella.
  - Permiten que los procesos se comuniquen de manera eficiente, eliminando la necesidad de almacenar datos intermedios en archivos en el sistema de archivos, lo que reduce el uso de recursos del sistema.

Características

- Al abrir una tubería con nombre, se le asocia un nombre de fichero de disco. Este fichero funciona como una tubería FIFO. El SO decide sonde se almacena este fichero
- Cuando se ejecuta el comando `mkfifo`, se crea un archivo especial FIFO en el sistema de archivos. Sin embargo, los datos escritos en un archivo FIFO no se almacenan en el disco como lo hacen los archivos regulares.
- Los datos escritos en un archivo FIFO se almacenan temporalmente en la memoria y se transmiten directamente a cualquier proceso que esté leyendo del archivo FIFO
- Una vez que los datos se leen del archivo FIFO, se eliminan de la memoria
- Cualquier proceso puede abrir este fichero (lectura o escritura). Solo se requiere que los procesos utilicen el mismo nombre de la tubería.

Ejemplo de tubería con nombre: escritor-lector



Ejemplo de tubería con nombre: escritor-lector

- Ver códigos fifo_write.c y fifo_read.c
  - `mkfifo` crea una tubería con nombre. Toma dos argumentos: el nombre del archivo FIFO que se va a crear y los **permisos** que se le van a asignar.
  - `open` abre la tubería (escritura o lectura).

Valor	Descripción
S_IRWXU	Lectura, escritura y ejecución para el propietario
S_IRUSR	Lectura para el propietario
S_IWUSR	Escritura para el propietario
S_IXUSR	Ejecución para el propietario
S_IRWXG	Lectura, escritura y ejecución para el grupo
S_IRGRP	Lectura para el grupo
S_IWGRP	Escritura para el grupo
S_IXGRP	Ejecución para el grupo
S_IRWXO	Lectura, escritura y ejecución para otros
S_IROTH	Lectura para otros
S_IWOTH	Escritura para otros
S_IXOTH	Ejecución para otros

Escritor

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>

int main()
{
  int fd;

  mkfifo("myfifo", PERM_FILE);

  printf("Obrint pipe per escriptura\n");
  fd = open("myfifo", O_WRONLY);
  printf("Escrivint missatge\n");
  write(fd, "test", 5);
  printf("Tancant\n");
  close(fd);
}
```

Lector

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>

int main()
{
  char buf[20];
  int fd;

  printf("Obrint pipe per lectura\n");
  fd = open("myfifo", O_RDONLY);
  printf("Llegint missatge\n");
  read(fd, buf, 5);
  printf("Rebut %s, tancant.\n", buf);
  close(fd);
}
```

Gracias

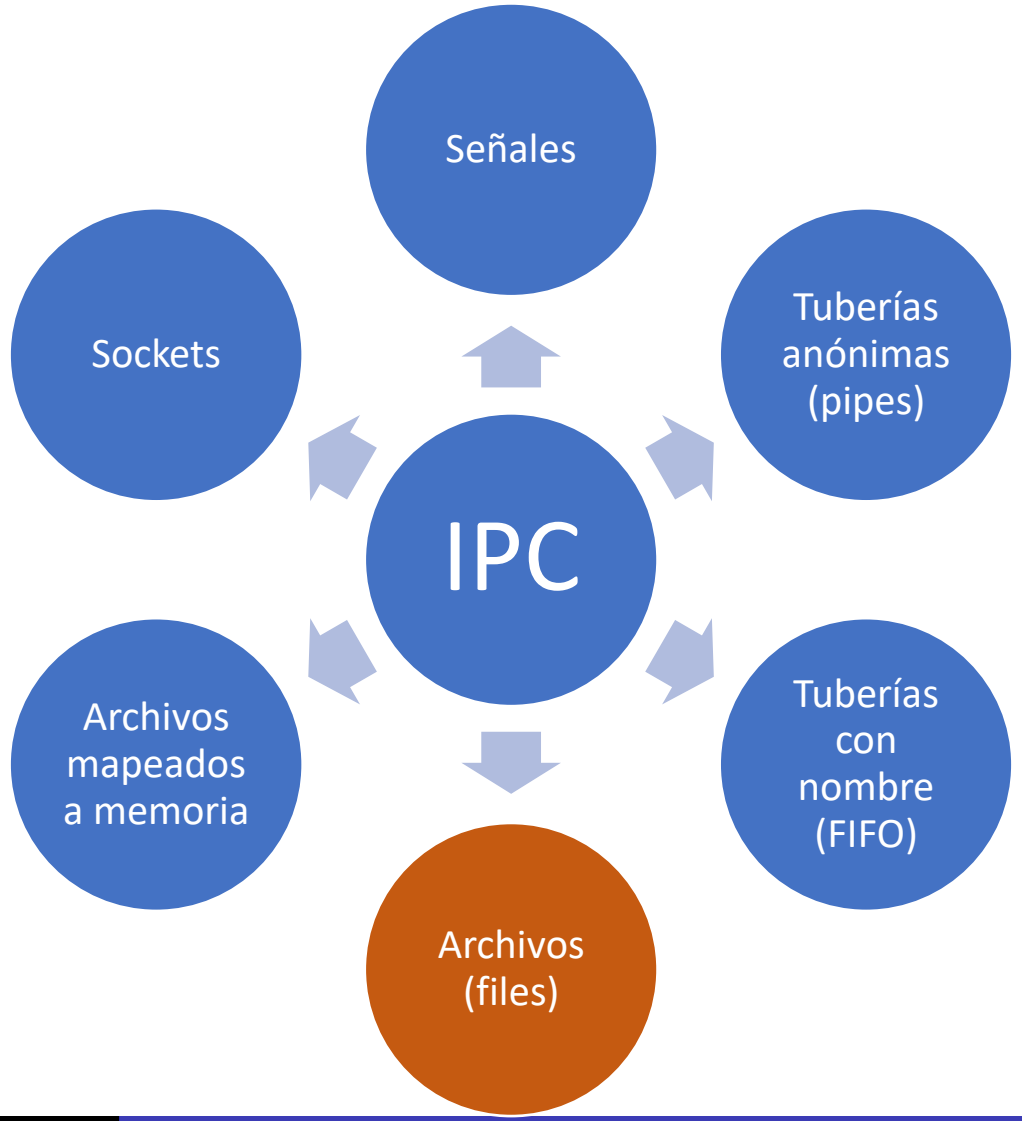
Comunicación entre procesos: Archivos

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Tipos de comunicación entre procesos



Definición

- Un **archivo regular** (*regular file*) es tipo de archivo que contiene datos almacenados en una secuencia de bytes, que se pueden leer y escribir.
- Pueden ser utilizados como un mecanismo de comunicación simple y efectivo entre procesos.
  - Uno de los procesos debe crear el archivo regular y establecer los permisos de acceso adecuados para permitir que otros procesos escriban y lean del archivo.
  - El proceso emisor escribe datos en el archivo regular, y el proceso receptor lee los datos.
- Esta técnica es especialmente útil para la comunicación entre procesos que no necesitan una alta tasa de transferencia de datos (rendimiento sujeto a la velocidad de acceso al disco).
- Otros tipos de archivos en los SO incluyen **directorios**, **archivos de dispositivo** (*device files*) o **enlaces simbólicos** (*symbolic links*).

Ventajas y desventajas

- **Ventajas**:
  - El disco es un dispositivo de almacenamiento al que pueden acceder todos los procesos.
  - El SO ofrece funciones para que un proceso (un puntero) se pueda situar en cualquier posición del archivo.
  - Múltiples procesos pueden acceder al mismo tiempo a un fichero (escritura/lectura)
- **Desventajas:**
  - Manipular archivos es más complicado que manipular directamente la memoria.
  - Hay que asegurar en todo momento que el fichero contienen datos consistentes en caso de que múltiples procesos puedan escribir/leer datos.

Funciones en C para acceder a ficheros (lectura/escritura)

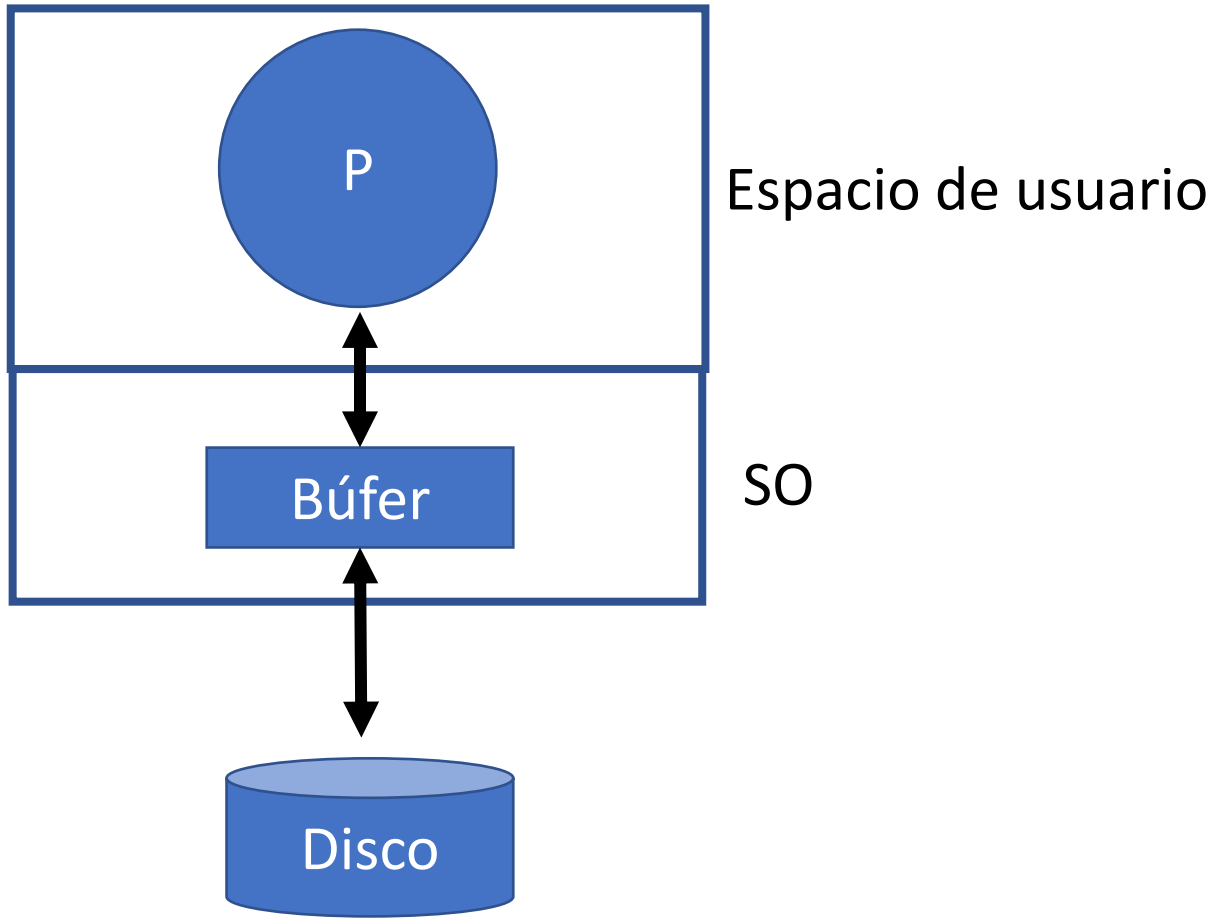
- **Llamadas al sistema:**
  - `open`
  - `read`
  - `write`
  - …
- **Funciones de librería estándar**
  - `fopen`
  - `fread`
  - `fwrite`
  - …

Llamadas al sistema para manipular ficheros

- `open` y `creat`: abre y crea un fichero en el disco. La función devuelve un entero (el *fd* del fichero abierto)
- `close` : Cierra un fichero. Hay que pasar por parámetro el *fd* del fichero a cerrar.
- `read` y `write` : lee y escribe datos (p. ej. Bytes) de/a cualquier fichero. Hay que pasarle el *fd* como parámetro.
- `lseek`: establece la posición actual dentro de un fichero en el disco (lectura/escritura). Necesita *fd*.

Llamadas al sistema para manipular ficheros

- El SO utiliza un búfer interno para aumentar la eficiencia en el acceso a disco



Llamadas al sistema para manipular ficheros

write_char_int.c

```c
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>

void main(void)
{
  int i, fd;
  char *a = "lluis";

  fd = open("file.data",
        O_WRONLY | O_CREAT | O_TRUNC,
        S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);

  for(i = 0; i < 100; i++)
  {
    write(fd, a, 5);
    write(fd, &i, sizeof(int));
  }

  close(fd);
}
```

read_char_int.c

```c
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>

void main(void)
{
  int i, k, fd;
  char a[6];

  fd = open("file.data", O_RDONLY);

  for(i = 0; i < 100; i++)
  {
    read(fd, a, 5);
    read(fd, &k, sizeof(int));

    a[5] = 0; // Equivalent a[5] = '\0'
    printf("Llegit: %s y %d\n", a, k);
  }

  close(fd);
}
```

Llamadas al sistema para manipular ficheros

write_vector_int.c

```c
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>

void main(void)
{
  int i, fd, vector[1000];

  fd = open("file.data",
        O_WRONLY | O_CREAT | O_TRUNC,
        S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);

  for(i = 0; i < 1000; i++)
     vector[i] = 2 * i + 1;

  write(fd, &vector, sizeof(int) * 1000);

  close(fd);
}
```

read_vector_int.c

```c
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>

void main(void)
{
  int i, fd, vector[1000];

  fd = open("file.data", O_RDONLY);

  read(fd, vector, sizeof(int) * 1000);

  close(fd);

  for(i = 0; i < 1000; i++)
     printf("%d\n", vector[i]);
}
```

Llamadas al sistema para manipular ficheros

- `open, read, write`, … son funciones que llaman directamente al SO
  - El SO tiene un búfer interno (para cada fichero) que es accesible por todos los procesos.
  - `write` escribe datos al búfer interno del SO y devuelve enseguida. La operación de escritura a disco se realiza más tarde.
  - `read` comprueba primero si hay datos en el búfer interno del SO. Si no hay datos disponibles, se leen al disco. Mientras tanto, el proceso que hace el `read` queda en estado “bloqueado”.
  - Si un proceso escribe a fichero y otro lee después, es leerán los datos correctos aunque los datos no se hayan escrito todavía a disco.

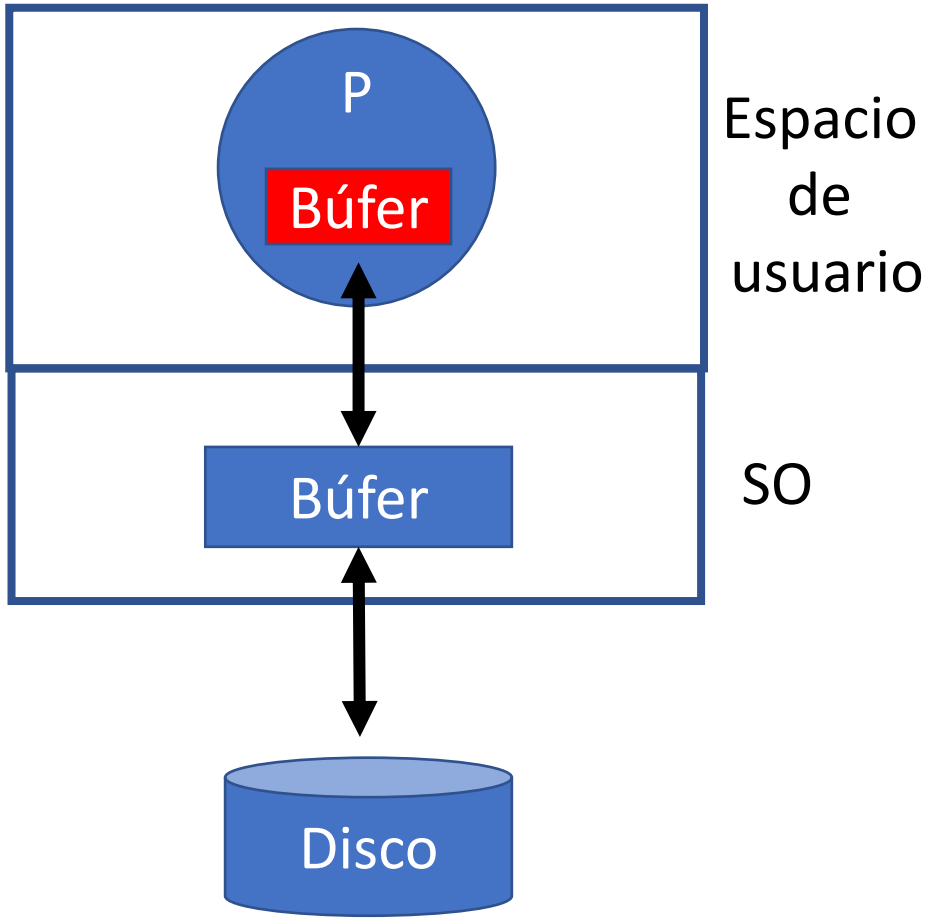
Librería estándar

- La **librería estándar "stdio"** nos ofrece las siguientes funciones para manipular ficheros en disco.
  - `fopen`: abre fichero de disco (lectura, escritura,..).
  - `fclose` : cierra el fichero.
  - `fread` y `fwrite` : lee y escribe datos (p. ej. Bytes). Internamente se llama a la función `read` y `write` del SO.
  - `fscanf` y `fprintf` : Lee y escribe datos en formato ASCII. Internamente se llama a `read` y `write` del SO.
  - `fseek` : define la posición actual dentro de un archivo.

Librería estándar

- Una **estructura FILE** es utilizada para representar un archivo abierto. La estructura FILE contiene información sobre el estado del archivo, como la posición actual del cursor de lectura/escritura y si el archivo se abrió para lectura, escritura o ambos.

```
typedef struct {
    int _file;        // Descriptor de archivo
    unsigned char *_ptr;  // Puntero al siguiente carácter a
leer/escribir
    unsigned char *_base; // Puntero al buffer subyacente
    int _cnt;         // Cantidad de caracteres en el buffer
    int _flag;        // Modo de apertura del archivo y otros flags
    int _fileid;      // Identificador único del archivo
} FILE;
```



- Las funciones de la librería estándar realizan internamente las llamadas a sistema cuando sea necesario.

Librería estándar

fwrite_char_int.c

```c
#include <stdio.h>

void main(void)
{
  FILE *fp;
  int i;

  char *a = "lluis";

  fp = fopen("log_fopen.data", "w");

  for(i = 0; i < 100; i++)
  {
    fwrite(a, sizeof(char), 5, fp);
    fwrite(&i, sizeof(int), 1, fp);
  }

  fclose(fp);
}
```

fread_char_int.c

```c
#include <stdio.h>

void main(void)
{
  FILE *fp;
  int i, k;
  char a[6];

  fp = fopen("log_fopen.data", "r");

  for(i = 0; i < 100; i++)
  {
    fread(a, sizeof(char), 5, fp);
    fread(&k, sizeof(int), 1, fp);

    a[5] = 0; // Equivalent a[5] = '\0'
    printf("Llegit: %s y %d\n", a, k);
  }

  fclose(fp);
}
```

Librería estándar

fprintf_char_int.c

```c
#include <stdio.h>

void main(void)
{
  FILE *fp;
  int i;

  char *a = "lluis";

  fp = fopen("log_fopen.data", "w");

  for(i = 0; i < 100; i++)
  {
    fprintf(fp, "%s\n", a);
    fprintf(fp, "%d\n", i);
  }

  fclose(fp);
}
```

fscanf_char_int.c

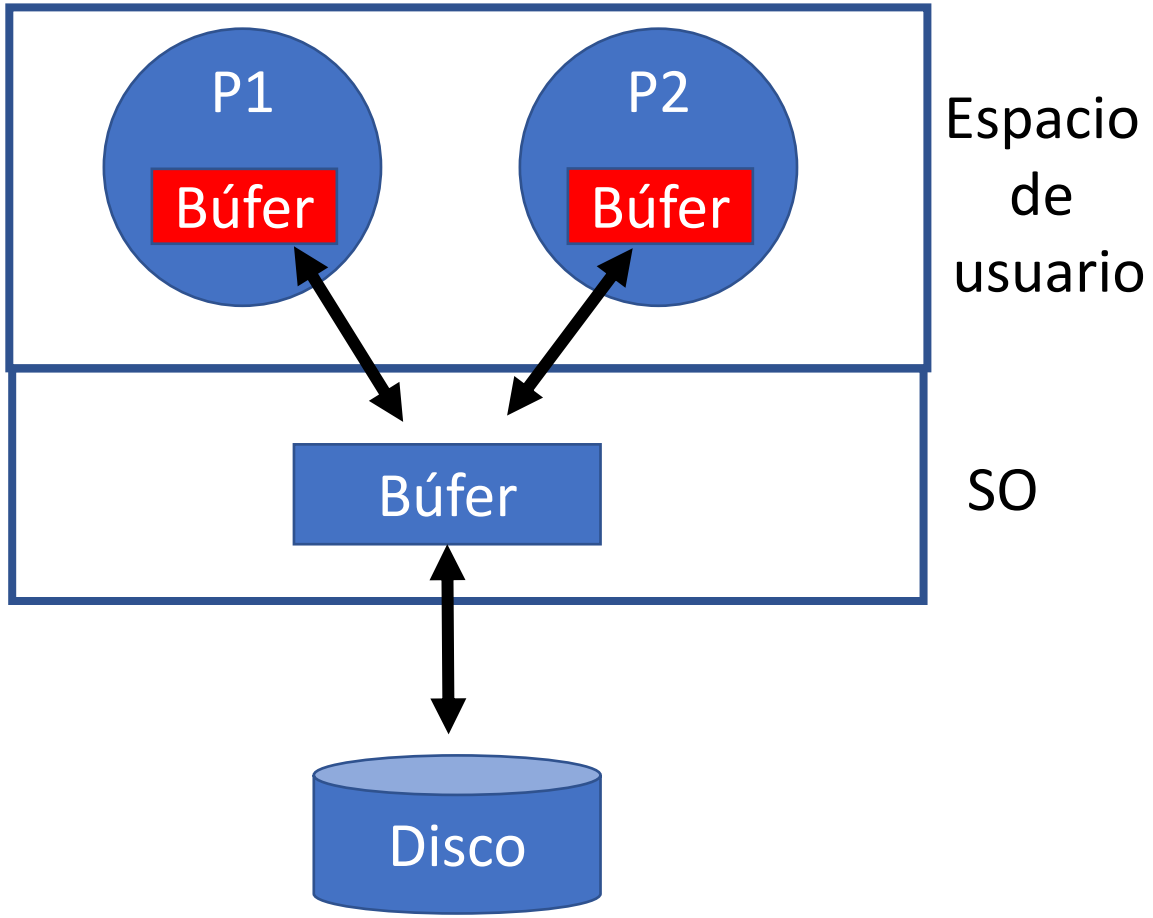
```c
#include <stdio.h>

void main(void)
{
  FILE *fp;
  int i, k;
  char a[6];

  fp = fopen("log_fopen.data", "r");

  for(i = 0; i < 100; i++)
  {
    fscanf(fp, "%s", a);
    fscanf(fp, "%d", &k);

    printf("Llegit: %s y %d\n", a, k);
  }

  fclose(fp);
}
```

Librería estándar

- La **estructura FILE** y las funciones `fopen`, `fprintf`, `fscanf`, `fwrite`, `fread`,… son funciones de la **librería estándar "stdio"**
  - Utiliza búfer propio (no visible a otros procesos)
  - Las operaciones `fprintf` o `fwrite` escriben datos al búfer FILE . Cuando el búfer se llena, se escriben los datos en el disco utilizando la llamada al sistema `write` (que escribe en un búfer interno del SO)
  - Las operaciones `fscanf` o `fread` leen los datos del búfer FILE. Si no hay datos disponibles, se realiza la llamada al sistema `read` para llenar el búfer FILE. Mientrastanto, el proceso queda bloqueado.
  - La librería estándar consigue así minimizar el uso de las funciones `write` y `read` (llamadas a sistema)

Librería estándar

- En algunos casos es posible que este método de comunicación entre procesos no sea el más adecuado.
- Múltiples procesos (cada uno con su propio búfer). Un dato no se verá de forma inmediata por otro proceso.



Gracias

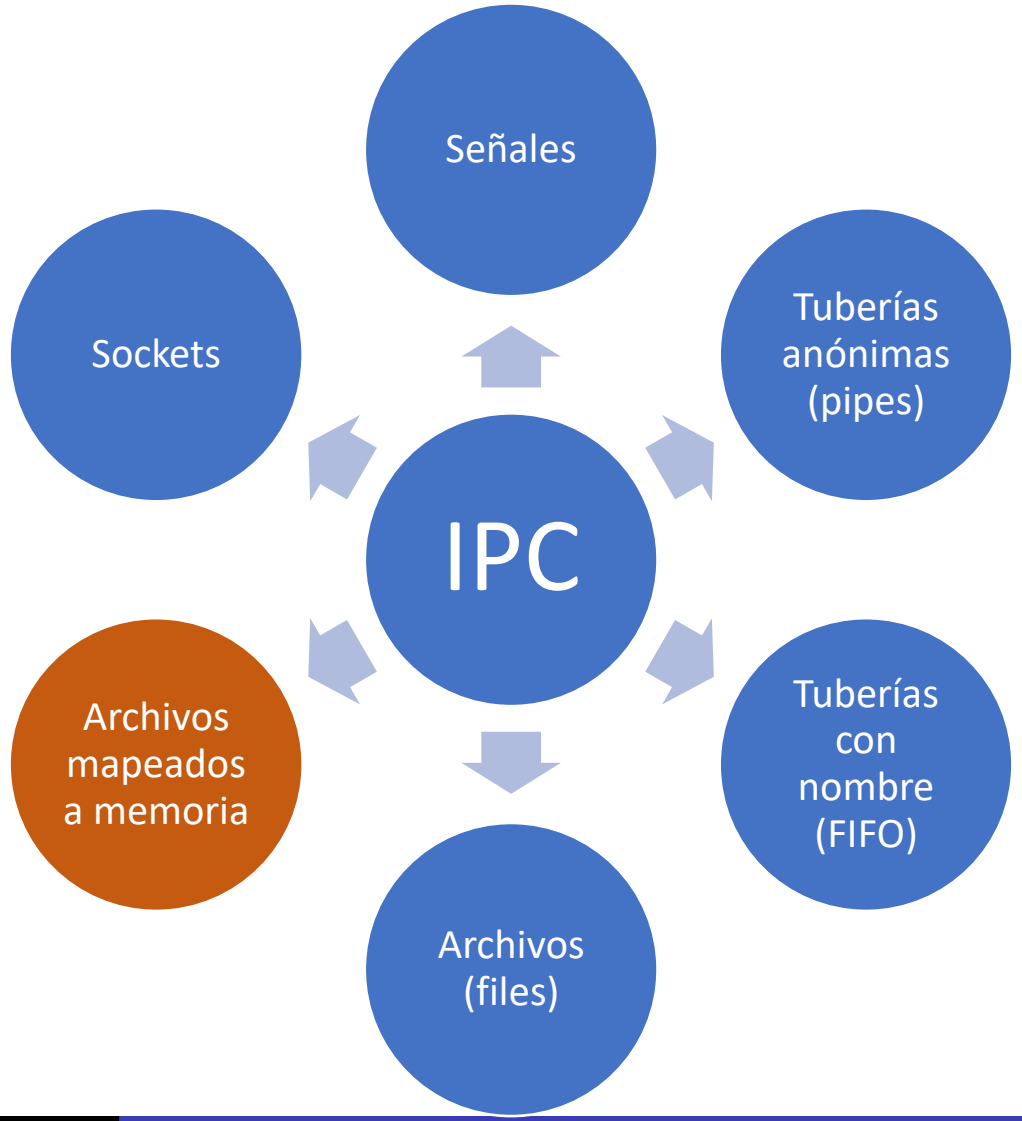
Comunicación entre procesos: Archivos mapeados a memoria

Sistemas Operativos I

Oliver Díaz

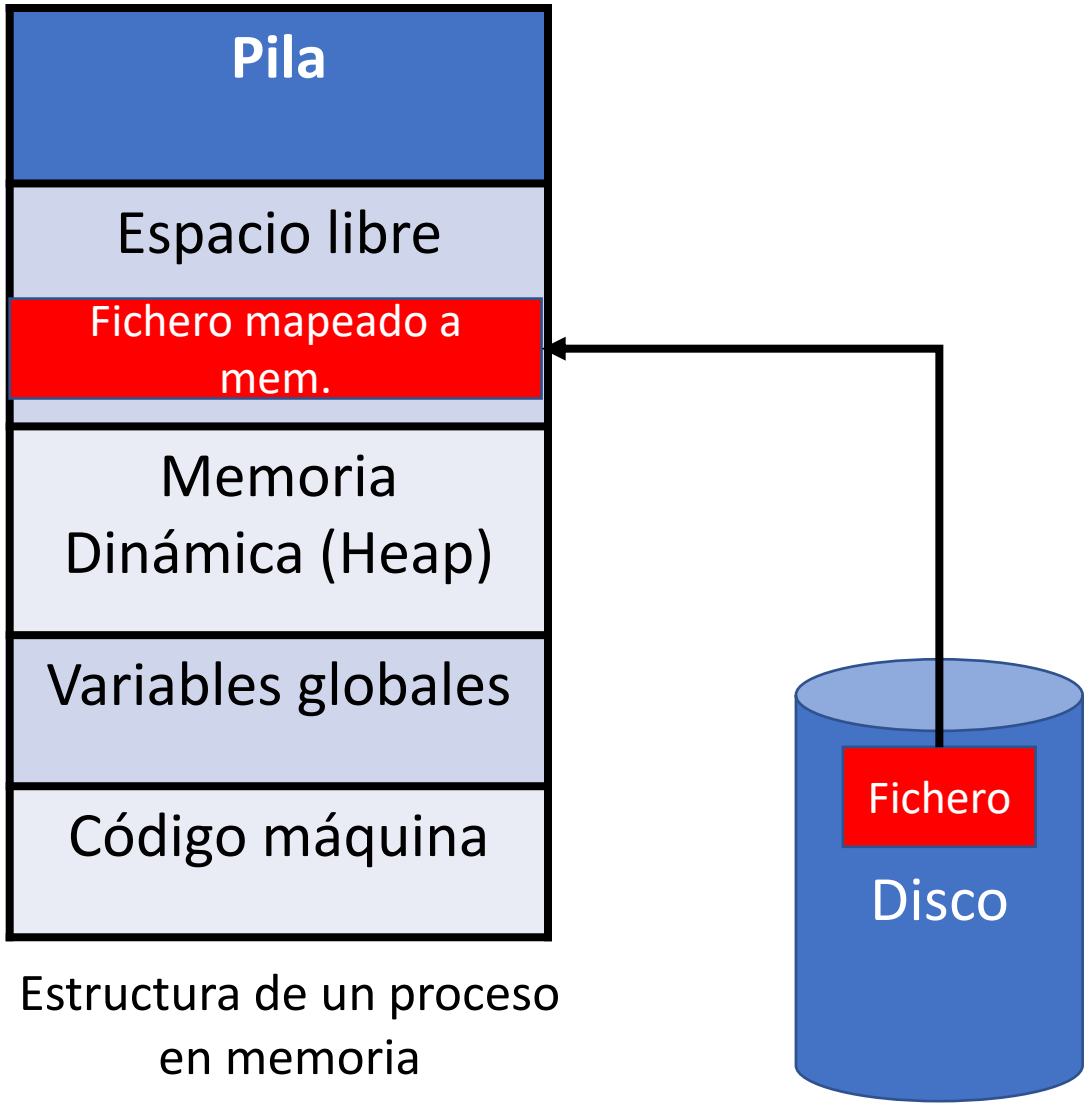
Grau d'Enginyeria Informàtica

Tipos de comunicación entre procesos



Definición

- Permite que dos (o más) procesos mapeen el contenido de un archivo en memoria, permitiendo que los procesos puedan acceder a él como si fuera una parte de la memoria del proceso.
- El archivo mapeado a memoria es un archivo que se asocia a una región contigua de memoria virtual.
- Los cambios realizados por un proceso en un archivo mapeado son inmediatamente visibles para todos los procesos que comparten el mismo archivo mapeado a memoria.



Estructura de un proceso en memoria

mmap

- Para utilizar archivos mapeados a memoria, los procesos primero deben crear o abrir el archivo en el sistema de archivos: p.ej. `open()`
- A continuación, los procesos pueden utilizar la función `mmap()` para asignar una región de memoria virtual del proceso al archivo.
- `mmap()` devuelve un puntero a la región de memoria virtual asignada, que se puede utilizar para leer y escribir datos en el archivo

```
void * mmap (void *address, size_t length, int protect, int flags, int fd, off_t offset)
```

mmap (atributos)

```
void * mmap (void *address, size_t length, int protect, int flags, int fd, off_t offset)
```

- **address**: especifica la dirección en la que se desea que comience el mapeo. Si se establece en 0, el SO elige la dirección automáticamente.
- **length**: especifica la longitud del mapeo en bytes.
- **protect**: especifica los permisos de protección de la memoria mapeada. Se pueden establecer permisos de lectura, escritura y/o ejecución.
- **flags**: especifica las opciones de configuración del mapeo, como si se desea que el mapeo sea compartido entre procesos o privado.
- **fd**: especifica el descriptor de archivo del archivo que se desea mapear. Debe haberse abierto previamente con la llamada a open().
- **offset**: especifica el desplazamiento en el archivo desde el cual se iniciará el mapeo.

Ventajas y desventajas

- **Ventajas**:
  - Acceso rápido (**alta eficiencia**) al contenido de un fichero como si fuera un vector de datos sin necesidad de utilizar las funciones read y write (Ver mmap.c)
  - Previene de errores y accesos no autorizados a los datos (**seguridad**)
  - El SO se encarga de cargar en memoria la parte necesaria del fichero para leerlo. Además, el SO se encarga de guardar la parte necesaria del fichero si éste cambia.
  - El SO la utiliza para implementar la **memoria virtual** de los procesos.
- **Desventajas:**
  - Limitaciones en el tamaño del archivo (depende de la cantidad de memoria disponible en el sistema)
  - Puede ser más complicado que otros mecanismos de comunicación entre procesos.

Ejemplo

- Mecanismo muy eficiente y fácil de comunicación entre procesos (con relación padre-hijo o no)

- fork-variables-malloc.c: Ejemplo de fork con un malloc que no comparten memoria entre procesos padre-hijo.
- fork-variables-mmap.c: Ejemplo equivalente que utiliza llamada a sistema mmap para compartir memoria padre-hijo.

Gracias

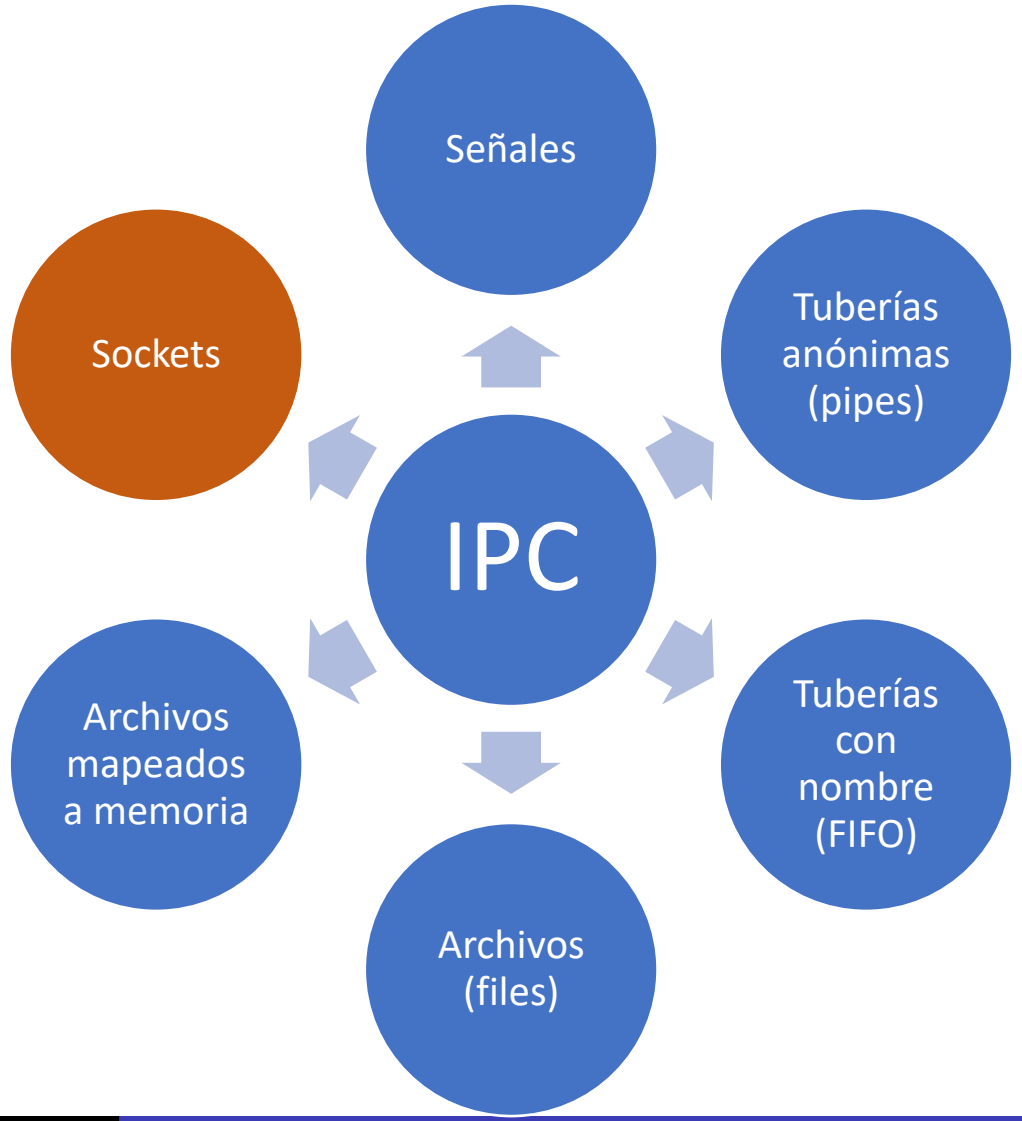
Comunicación entre procesos: Sockets

Sistemas Operativos I

Oliver Díaz

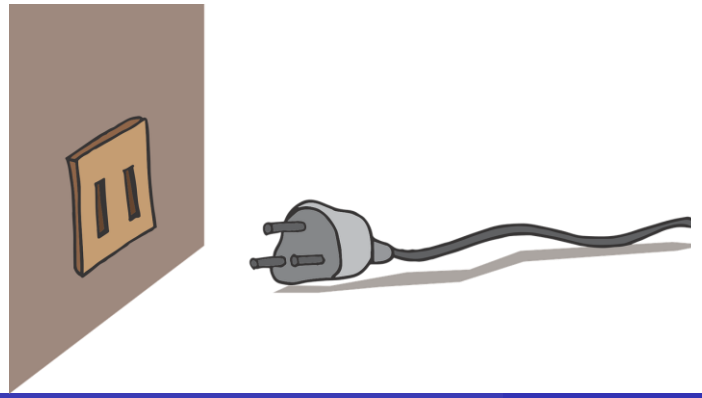
Grau d'Enginyeria Informàtica

Tipos de comunicación entre procesos



Definición

- Un **socket** es un objeto que permite la comunicación entre procesos (del mismo o diferentes ordenadores) a través de una red o un canal de comunicación local.
- Proporciona un mecanismo para enviar y recibir datos entre procesos, y se puede utilizar para implementar una amplia variedad de protocolos de comunicación, como TCP/IP, UDP, y UNIX domain sockets.
- La comunicación vía sockets es compleja (existe una asignatura dedicada a este tema: Software Distribuido)

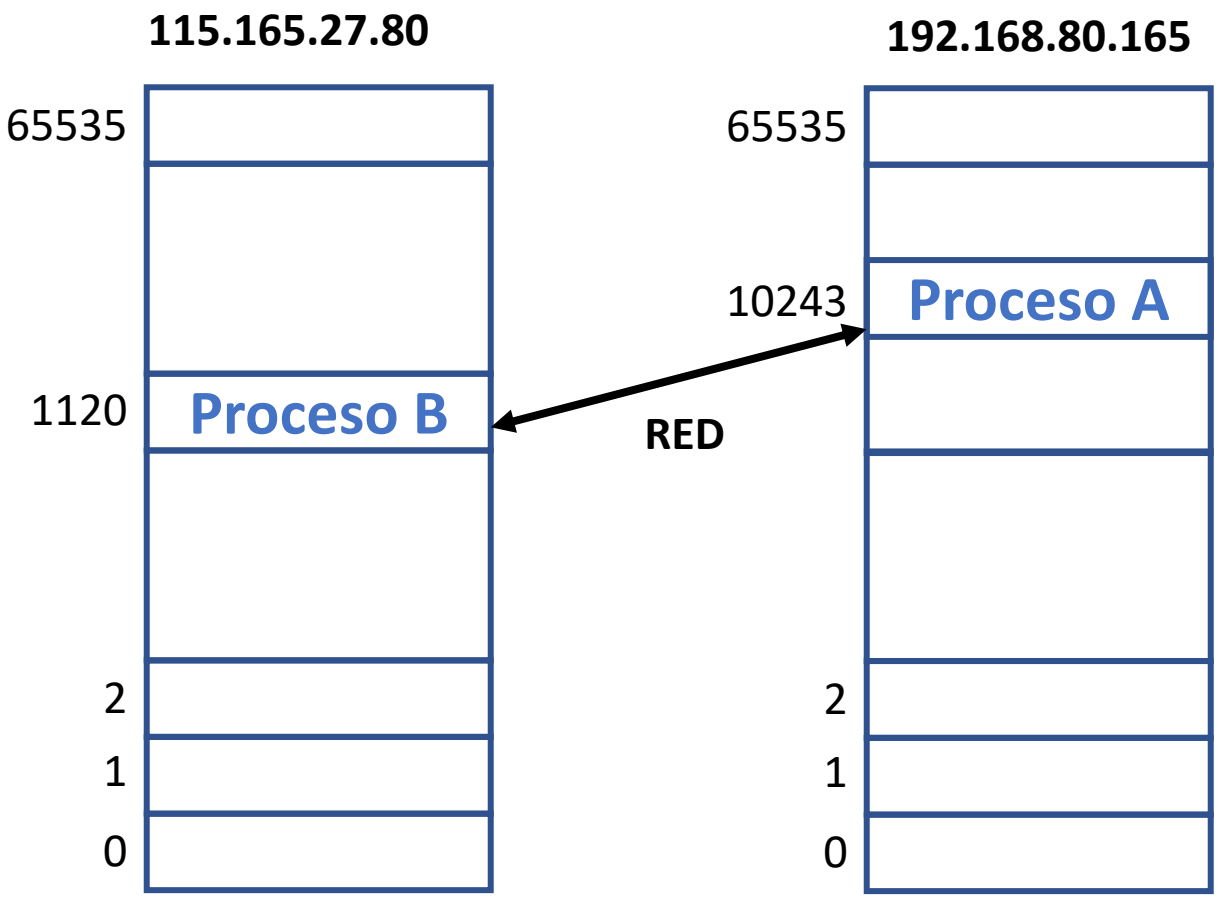


Pasos para la comunicación

- Los procesos primero deben crear un objeto de tipo socket utilizando la función `socket()`. Esta función devuelve un descriptor de archivo que se puede utilizar para enviar y recibir datos a través del socket.
- Después de crear el socket, los procesos pueden utilizar las funciones `bind()` y `listen()` para asignar una dirección al socket y esperar conexiones entrantes.
- Cuando un cliente desea conectarse al servidor, utiliza la función `connect()` para establecer una conexión con el servidor.
- Una vez que se ha establecido la conexión entre el cliente y el servidor, se pueden utilizar las funciones `send()` y `recv()` para enviar y recibir datos a través del socket.

Conceptos

- Un ordenador se identifica mediante una **dirección IP** (192.168.80.165)
- Un proceso A puede escuchar las peticiones entrantes y engancharse a un **puerto** (p.ex. 10243)
- Cualquier otro proceso B puede enviar un mensaje al proceso A
  - Se abre un canal de comunicación bilateral con la dirección IP y el puerto
  - Si B escribe un mensaje, este llegará a A (y viceversa).



Ejemplo

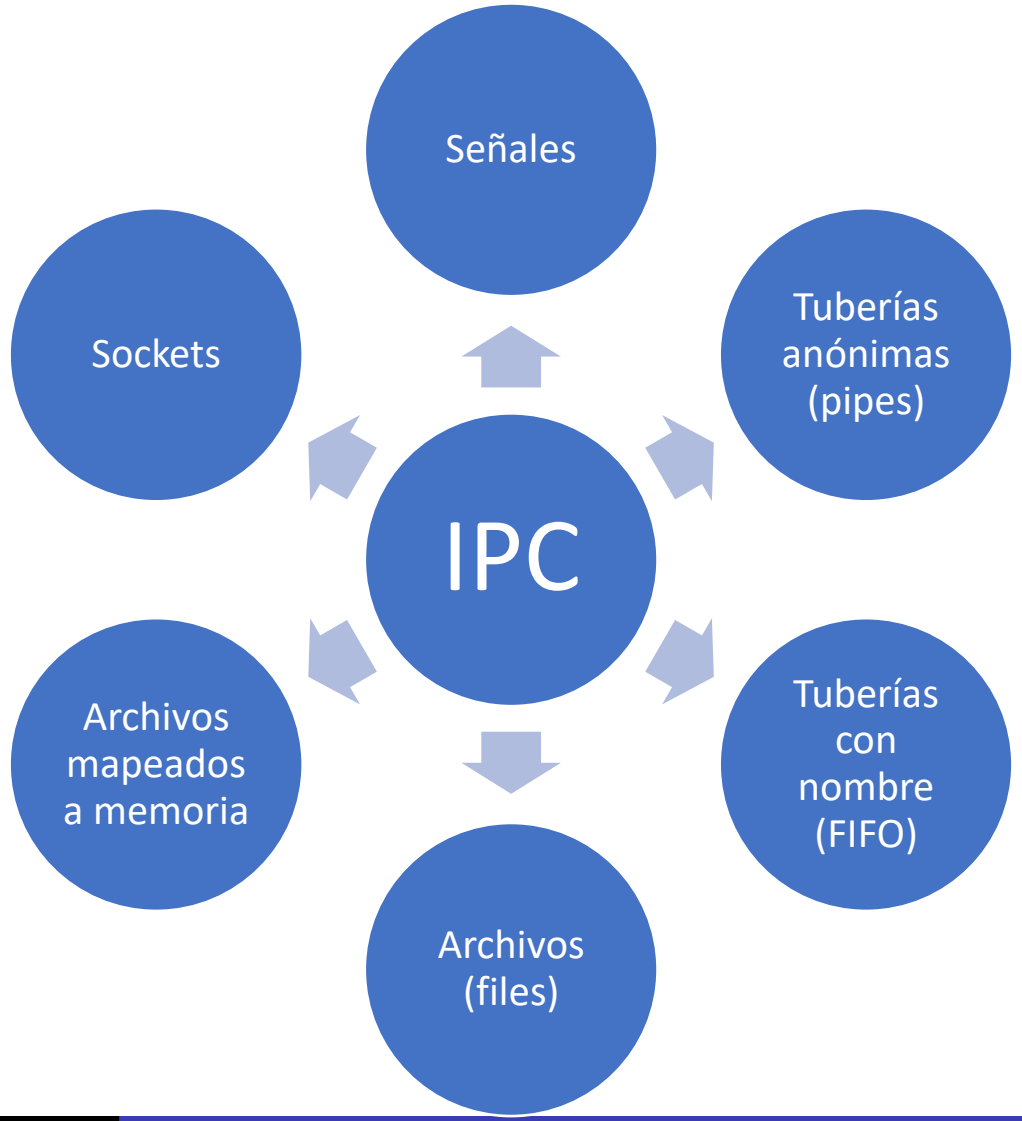
- socket_server.c : servidor (escucha peticiones entrantes)
- socket_cliente.c: cliente (hace la petición de conexión con el servidor)

- En la asignatura de software distribuido tendréis todos los detalles (Java)

Aplicaciones

- Ofrecen una gran flexibilidad y son muy utilizados para implementar aplicaciones de red, como servidores web, clientes de correo electrónico y aplicaciones de mensajería instantánea.
- Los sockets también se pueden utilizar para la comunicación entre procesos en la misma computadora, utilizando **UNIX domain sockets**.
  - Funcionan a través de un archivo especial en el sistema de archivos en lugar de una dirección IP y un número de puerto, como lo hacen los sockets de red convencionales.
  - Útiles para aplicaciones que requieren una comunicación de alta velocidad y baja latencia, como servidores web, bases de datos y aplicaciones multimedia.

Resumen de la unidad



Gracias